Variational Auto-Encoders (VAEs) and the Evidence Lower-BOund (ELBO)

This practical

- provides skeleton code for a VAE – you are tasked with filling in the missing bits and pieces
- has a couple of experiments to visualize interesting/useful aspects of the VAE
- is implemented using the python library JAX
- will give a conceptual, rather than technical, introduction to the ELBO and variants thereof

Before getting started, let us recap the VAE.

Background

VAE was simultaneously proposed by Diederik P. Kingma and Max Welling [1] and Danilo Jimenez Rezende, Shakir Mohamed and Daan Wierstra [2] independently. The former was published slightly prior to the latter, and has recieved considerably more citations. If you have not yet read/looked at either, I recommend checking out [1], as it will help you in this practical (and, of course, since it is a great piece of work).

Essentially, a VAE can be seen as a mapping from an input space to a latent (hidden or unobserved) space, then from the latent space to an output space. It consists of two parts, called encoder and decoder. With the help of encoder, we can map a given input sample $x_{ori} \sim p(x)$ to a distribution over the latent space. The distribution is denoted by $q_\phi(z|x_{ori})$ and ϕ denotes the parameters of the encoder (optimized via training), called (variational) parameters.

In mathematics, finding the best function (the distribution over the latent space in our case, i.e. $q_\phi(z|x_{ori})$) is done under the name of calculus of variations and this name also comes from that mathematical background. Variational approximation, (finding the best or closest function to a given function, in our case the true distribution over the latent sapce $p(z|x_{ori})$) is usually done because of the complexity of the original function (it means, it is hard to work with $p(z|x_{ori})$ because it has interactable terms).

So the idea of variational approximation is that to fix a family of distributions, say Q (think of it as the set of all possible normal distributions $Q = \{\mathcal{N}(\mu, \sigma)|\mu \in \mathbb{R}, \sigma \in \mathbb{R}^+\}$), and then approximate a complex function, needless to say $p(z|x_{ori})$, with a simpler function $q_\phi(z|x_{ori}) \in Q$. To this end, we need to find the closest $q_\phi(z|x_{ori})$ to $p(z|x_{ori})$, and the parameters of $q_\phi(z|x_{ori})$ (in our example, a good μ and σ) are called **variational parameters**. The subscript ϕ, also called variational paramters or encoder paramters, denotes that the distribution we get, depends on some other variables that has to be optimized (i.e. the variables that we use to map a given input x to a distribution over the latent space, for example $\phi = \{W_1, W_2, b_1, b_2\}$ if we are using a two layer MLP).

Note that $q_\phi(z|x)$ is a function of two variables both x and z and when you plug in some specific x, you still have a function of z. Therefore, $q_\phi(z|x)$ gives us the paramters of a distribution (in our example it gives us some specific μ and σ after plugging it x). Then, after getting the paramters of a distribution over the latent space $q_\phi(z|x_{ori})$, it is time to reconstruct the input sample x_{ori} from its latent representation $q_\phi(z|x_{ori})$, but the problem is that we can not use a function as input into the decoder! So what we can do is to draw (several) samples from the latent distribution $q_\phi(z|x_{ori})$ and recover a distribution over the input space. Another the problem is that it is that, sampling is not differentiable, making our model indifferentiable. To see why, try to diffrentiate a sample drawn from a Normal distribution, i.e. while $x \sim \mathcal{N}(\mu, \sigma)$ try to compute the following:

$$\frac{dx}{d\mu}$$

To solve the problem regarding the sampling and indifferentiability, [1] have proposed a trick called **reparameterization trick**, of which you'll hear more in the rest of this practical. But for now, it suffices to assume that we know we can use a sample like $z_{ori} \sim q_\phi(z|x_{ori})$, to map it back to the input sapce again, using a decoder, or so called **the generative** model. In our example, it is as simple as $z_{ori} \sim \mathcal{N}(\mu_{ori}, \sigma_{ori})$ where the μ_{ori} and σ_{ori} is obtained simply by plugging in x_{ori} into the encoder, i.e. $q_\phi(z|x_{ori})$.

As the decoder is acting on samples of a distribution, and becomes stochastic by virtue of sampling, it defines a distribution over the input space. Therefore, the **distribution recovered** from z_{ori} by the decoder, is denoted by $p_\theta(x|z_{ori})$.

The θ subscript is the parameters of the generative model, also called generative paramters are optimized jointly with ϕ. Finally, we can *decode* z_{ori} by sampling from $p_\theta(x|z_{ori})$. The decoding of z_{ori}, i.e. $x_{rec} \sim p_\theta(x|z_{ori})$, is commonly referred to as the *reconstruction*.

Note that in the original paper you do not see any distinction between x_{ori}, x_{rec} and x as they are in the same space, but it is added here to make it easy to explain the idea! Following [1], we avoid the use of subscripts in the rest of the practical.

Why?

There are plenty of interesting tasks that can be tackled using VAEs. Some important examples include dimensionality reduction (encoding x as z, such that dim(z) < dim(x)), noise filtering (a.k.a. reconstructing the input), generating synthetic data (sampling from $p_\theta(x|z)$) and density

estimation. Let us zoom in on the latter.

Say we are interested in learning the posterior distribution $p(z|x)$ for whatever reason. In most interesting scenarios, however, we cannot learn it since it is intractable. The intractability of

$$p(z|x) = \frac{p(z,x)}{p(x)}$$

stems from not being able to compute $p(x) = \int p(z,x)dz$ as it involves marginalizing over all z. Instead of learning $p(z|x)$ directly, we have to approximate it.

How?

The VAE is a form of (amortized) variational inference (VI). In VI we approximate the intractable posterior using a simpler distribution, $q_\phi(z|x)$, inferring ϕ by estimating the marginal log-likelihood, $\log p(x)$. Consequently, we actually estimate two densities.

The inference is performed by *maximizing* the **E**vidence **L**ower **BO**ound (ELBO in short), $\mathcal{L}$:

$$\mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p(z,x)}{q_\phi(z|x)}\right] \triangleq \mathcal{L} = -D_{KL}(q_\phi(z|x)||p(z)) + \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x|z)\right]$$

where $D_{KL}(q_\phi(z|x)||p(z))$ is the Kullback-Leibler (KL) divergence. The KL divergence is strictly non-negative, and zero if and only if $q_\phi(z|x) = p(z)$, while the marginal log-likelihood, $\log p(x)$, is a constant. It is also possible to write:

$$\log p(x) = D_{KL}(q_\phi(z|x)||p(z|x)) + \mathcal{L}$$

The $D_{KL}(q_\phi(z|x)||p(z|x))$ term is called **the variational gap** and is non-negative, so it can be seen that we have:

$$\log p(x) \geq \mathcal{L}$$

Therefore, the ELBO is a lower bound on the marginal log-likelihood and maximizing $\mathcal{L}$ indirectly minimizes $D_{KL}(q_\phi(z|x)||p(z|x))$ (which we do not have), giving us a better approximation of the intractable posterior. See [1] for the derivation. Note that the negated ELBO is often referred to as the negative log-likelihood (NLL).

In this practical, as in [1], we will assume that $q_\phi(z|x) = \mathcal{N}(z;\mu,\sigma^2)$, $p(z) = \mathcal{N}(0,1)$ and $p_\theta(x|z) = \text{Bernoulli}(x;\theta)$. Hence we can evaluate all terms in the ELBO, where $p(z,x) = p_\theta(x|z)p(z)$.

Benchmarking VAEs in Density Estimation Tasks

This will be a recurring topic in this practical. Namely, you may train on whichever objective function you deem fitting, however when reporting your results in order to compare the performance of your algorithm, the evaluation of the **correct** objective is crucial. For instance, if you want to compare your VAE's NLL scores with others', then make sure the objective function is in fact a lower-bound on the marginal log-likelihood. More on that later.

VAE in JAX

Now, let's import the necessary libraries. Feel free to add imports as you wish.

```
import os
import time
from tqdm import tqdm

import matplotlib.pyplot as plt
from tensorflow_probability.substrates import jax as tfp
import jax
import jax.numpy as jnp

from jax import jit, grad, lax, random
from jax.example_libraries import optimizers
from jax.example_libraries import stax
from jax.example_libraries.stax import Dense, FanOut, Relu, Softplus, Sigmoid
import numpy as np
from torch.utils import data
from torchvision.datasets import MNIST, FashionMNIST
from functools import partial

from jax.config import config
# IMPORTANT NOTE:
# if you have got a NaN loss and/or have trouble debugging. Then, set
# jax_disable_jit to True. This will help you print out the variables.
config.update('jax_disable_jit', False)
```

For the sake of simplicity, we use the MNIST dataset in this practical.

The next cell creates a PyTorch dataloader. Using either PyTorch or TensorFlow dataloaders seems to (currently) be the practice when working with JAX. The code has been slighlty altered after being copying from here.

```
def numpy_collate(batch):
  if isinstance(batch[0], np.ndarray):
    return np.stack(batch)
  elif isinstance(batch[0], (tuple,list)):
    transposed = zip(*batch)
    return [numpy_collate(samples) for samples in transposed]
  else:
    return np.array(batch)

class NumpyLoader(data.DataLoader):
  def __init__(self, dataset, batch_size=1,
               shuffle=False, sampler=None,
               batch_sampler=None, num_workers=0,
               pin_memory=False, drop_last=False,
               timeout=0, worker_init_fn=None):
    super(self.__class__, self).__init__(dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        sampler=sampler,
        batch_sampler=batch_sampler,
        num_workers=num_workers,
        collate_fn=numpy_collate,
        pin_memory=pin_memory,
        drop_last=drop_last,
        timeout=timeout,
        worker_init_fn=worker_init_fn)

class FlattenAndCast(object):
  def __call__(self, pic):
    return np.ravel(np.array(pic, dtype=jnp.float32))


batch_size = 245
# Define our dataset, using torch datasets
mnist_dataset = MNIST('/tmp/mnist/', download=True, transform=FlattenAndCast())
training_generator = NumpyLoader(mnist_dataset, batch_size=batch_size, num_workers=0)
mnist_dataset_test = MNIST('/tmp/mnist/', download=True, train=False)
#test_images = jnp.array(mnist_dataset_test.test_data.numpy().reshape(len(mnist_dataset_test.test_data), -1), dtype=jnp.float32)
test_images = jnp.array(mnist_dataset_test.data.numpy().reshape(len(mnist_dataset_test.data), -1), dtype=jnp.float32)

#test_labels = jnp.array(mnist_dataset_test.test_labels)
test_labels = jnp.array(mnist_dataset_test.targets)
```

Initializing the Network and Optimizer

Next we put together a simple VAE, using two-layered MLPs with 300 hidden units for both the encoder and decoder networks. Note that the two parallel ultimate layers in the encoder net output latent_dimensions-dimensional arrays. **Based on the respective ranges of these two outputs, which one is μ and which is σ?**

Also, note that we apply **the sigmoid function on the outputs of the decoder**. Recall that $p_\theta(x|z)$ models the paramters of a Bernoulli distribution.

Initialize the networks and the Adam optimizer.

```
latent_dimensions = 2

# encoder defs
encoder_init, encode = stax.serial(
    Dense(300), Relu,
    Dense(300), Relu,
    FanOut(2),
    stax.parallel(Dense(latent_dimensions), stax.serial(Dense(latent_dimensions), Softplus)),
)

# decoder defs
decoder_init, decode = stax.serial(
    Dense(300), Relu,
    Dense(300), Relu,
    Dense(28 * 28), Sigmoid
)


# initialize networks and get params
enc_init_key, dec_init_key = random.split(random.PRNGKey(100))
_, encoder_params = encoder_init(enc_init_key, input_shape=(-1, 28 * 28))
```

```
_, decoder_params = decoder_init(dec_init_key,
                                 input_shape=
                                 (-1, latent_dimensions))
params = encoder_params, decoder_params

# initialize optimizer
opt_init, opt_update, get_params = optimizers.adam(step_size=0.001)
opt_state = opt_init(params)
```

Task 1: Reparameterization Trick

See [1] to figure out what **reparameterization trick** is, and implement the reparameterization trick in the following function.

####### **The paper

an alternative method for generating samples from $q_\varphi(z|x)$

Let z be a continuous random variable, and $z \sim q_\varphi(z|x)$ be some conditional distribution

It is then often possible to express the random variable z as a deterministic variable $z = g_\phi(\epsilon, x)$, where ϵ is an auxiliary variable with independent marginal $p(\epsilon)$, and $g_\varphi(.)$ is some vector-valued function parameterized by ϕ.

This reparameterization is useful for our case since it can be used to rewrite an expectation w.r.t $q_\varphi(z|x)$ such that the Monte Carlo estimate of the expectation is differentiable w.r.t. φ

Given the deterministic mapping $z = g_\varphi(\epsilon, x)$ we know that

$$q_\varphi(z|x)\Pi_i dz_i = p(\epsilon)\Pi_i d\epsilon_i$$

$dz = \Pi_i dz_i$

Therefore

$$\int q_\varphi(z|x)f(z)dz = \int p(\epsilon)f(z)d\epsilon = \int p(\epsilon)f(g_\varphi(\epsilon, x))d\epsilon$$

a differentiable estimator can be constructed:

$$\int q_\varphi(z|x)f(z)dz = \frac{1}{L}\sum(l = 1tol)f(g_\varphi(x, \epsilon^l))$$

where $\epsilon \sim p(\epsilon)$

```
def sample(rng, mu, std):
  # use the reparameterization trick
  epsilon = random.normal(rng, mu.shape)

  # Transform ε using the parameters mu and log_std
  z = mu + jnp.exp(std / 2) * epsilon  # log_std is half the log variance (std deviation)

  # Apply a sigmoid function to constrain the output to [0, 1] for Bernoulli data
  z = jax.nn.sigmoid(z)
  return z
```

Continuing with the VAE pipeline. "@jit" is a JAX decoration which apparently heavily speeds up operations. Beware, however, that it does not support all operations, putting restrictions on the function it decorates.

```
@jit
def forward(params, x, rng):
    encoder_params, decoder_params = params
    phi = encode(encoder_params, inputs=x)
    z = sample(rng, mu=phi[0], std=phi[1])
    bernoulli_theta = decode(decoder_params, z)
    return phi, z, bernoulli_theta
```

Task 2: ELBO

Compute the ELBO in the following function. Feel free to add or remove input arguments, except for params; params should be the first input argument (it's due to how JAX computes gradients). Also, since the optimizer *minimizes* the objective function, make sure to return the negated ELBO. Hint: recall the definition of the ELBO, note that it can be Monte Carlo approximated as follows

$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x|z)p(z)}{q_\phi(z|x)}\right] \simeq \frac{1}{L}\sum_l \log \frac{p_\theta(x|z_l)p(z_l)}{q_\phi(z_l|x)}$,

and, following [1], let $L = 1$.

Finally, package `tfp.distributions` could be used to calculate the (log) probability of different distributions. For instance, we could use `tfp.distributions.Bernoulli` to compute the likelihood of the pixels.

```
@jit
def get_elbo(params, x, rng):
    # the negative elbo should be a scalar value, i.e. averaged over the mini-batch.
    (mu,sigma), z, bernoulli_theta = forward(params, x, rng)

    log_p_x_given_z = jnp.sum(x * jnp.log(bernoulli_theta) + (1 - x) * jnp.log(1 - bernoulli_theta))
    log_p_z = jnp.sum(-0.5 * z**2 - 0.5 * sigma - 0.5 * jnp.log(2 * jnp.pi))
    log_q_z_given_x = jnp.sum(-0.5 * ((z - mu) / jnp.exp(sigma))**2 - sigma - 0.5 * jnp.log(2 * jnp.pi))

    log_p_x_given_z = jnp.sum(x * jnp.log(bernoulli_theta + 1e-8) + (1 - x) * jnp.log(1 - bernoulli_theta + 1e-8)) ##stabilizing


    elbo = log_p_x_given_z + log_p_z - log_q_z_given_x
    neg_elbo = -jnp.mean(elbo)


    # write code to compute the ELBO
    return neg_elbo
```

Now to the training and evaluation steps. Note that we have normalized the pixel values in order for them to fit the probabilistic model.

```
def epoch_step(epoch, training_generator, opt_state):
    # an rng, random number generator, is required for all stochastic operations
    rng = random.PRNGKey(epoch)
    nlls = []
    for i, (x, y) in enumerate(tqdm(training_generator)):
        x = jax.device_put(x) / 255.0

        value, grads = jax.value_and_grad(get_elbo)(get_params(opt_state), x, rng)

        grads = jax.tree_map(lambda g: jnp.clip(g, -1.0, 1.0), grads) #######cliping added

        opt_state = opt_update(i, grads, opt_state)

        nlls.append(value)

    return opt_state, np.mean(nlls)
```

Luckily, the complete test set fits in the memory, and so we can compute the average NLL on all images at once without taking the batch size into account. This might not always be possible, in which case you must be careful regarding how you average the NLL scores, since all batches needn't be of the same size. Failing to do so will make your NLL scores unfit for benchmarking.

```
def evaluate(opt_state, x, test_rng):
    params = get_params(opt_state)
    x = jax.device_put(x) / 255.0
    nll = get_elbo(params, x, test_rng)
    return nll
```

** **bold text**Plot the NLL vs. epochs curves (train and test) and see that they're decaying.**

```
n_epochs = 20
test_key = random.PRNGKey(10)
for epoch in range(n_epochs):
    print("Epoch: ", epoch)
    opt_state, nll_train = epoch_step(epoch, training_generator, opt_state)
    nll_test = evaluate(opt_state, test_images, test_key)
    print(f"Train NLL: {nll_train}")
    print("Test NLL: ", nll_test, '\n')
```

```
Epoch:  0
100%|██████████| 245/245 [00:20<00:00, 11.71it/s]
Train NLL: nan
Test NLL:  nan

Epoch:  1
100%|██████████| 245/245 [00:18<00:00, 13.44it/s]
Train NLL: nan
Test NLL:  nan

Epoch:  2
100%|██████████| 245/245 [00:19<00:00, 12.54it/s]
Train NLL: nan
Test NLL:  nan

Epoch:  3
```

```
100%|██████████| 245/245 [00:18<00:00, 12.91it/s]
Train NLL: nan
Test NLL:  nan

Epoch:  4
100%|██████████| 245/245 [00:18<00:00, 12.97it/s]
Train NLL: nan
Test NLL:  nan

Epoch:  5
100%|██████████| 245/245 [00:18<00:00, 13.56it/s]
Train NLL: nan
Test NLL:  nan

Epoch:  6
100%|██████████| 245/245 [00:18<00:00, 13.07it/s]
Train NLL: nan
Test NLL:  nan

Epoch:  7
100%|██████████| 245/245 [00:17<00:00, 13.67it/s]
Train NLL: nan
Test NLL:  nan

Epoch:  8
100%|██████████| 245/245 [00:18<00:00, 12.91it/s]
Train NLL: nan
Test NLL:  nan

Epoch:  9
100%|██████████| 245/245 [00:17<00:00, 13.85it/s]
Train NLL: nan
Test NLL:  nan

Epoch:  10
100%|██████████| 245/245 [00:19<00:00, 12.77it/s]
Train NLL: nan
Test NLL:  nan

Epoch:  11
100%|██████████| 245/245 [00:17<00:00, 13.98it/s]
Train NLL: nan
```

Task 3: Experiments

Let's put the trained VAE to use by visulizing the latent space and sampling synthetic data. Complete the following code snippets to do so.
Visualize the latent space. Use the labels y to color the latent representations in a scatter plot.

Hint: Encode the training or test set, and use `plt.scatter`.

```
def show_latent_space():
  encoder_params = get_params(opt_state)[0]
  pass
```

Monitor the reconstruction quality of your model. Visualize a single input image together its reconstruction.

```
def reconstruct_an_image():
  x = test_images[0]  # the image to be reconstructed
  params = get_params(opt_state)
  # call the forwad function and use its output to reconstruct the image
```

Generate synthetic data. Hint: sample $z \sim p(z)$ or "walk" in the latent space. Decode the z you sample/visit and plot the decoded image. Think about what output you expect when you sample from 1) a Bernoulli distribution, or 2) regard the Bernoulli parameter as the sample.

```
@jit
def sample_image(decoder_params):
    # feel free to add other args to the function
    # either sample from the prior, or walk/do a grid search over z's
    # then use decoder_params to decode the latent samples
    return sampled_image
```

β-VAE

The β-VAE was proposed by Higgins et al. in 2017 and is a straightforward but important extension of the VAE. In terms of architecture, β-VAE and VAE are indistinguishable. In fact, the novelty in the work by Higgins et al. lies in a minor modification of the ELBO objective, the β-objective

$$\mathcal{L}_\beta = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \beta \mathbb{E}_{q_\phi(z|x)}[\log \frac{q_\phi(z|x)}{p(z)}],$$

where $\beta \geq 0$ is a hyperparameter weighting the KL divergence

$\text{KL}(q_\phi(z|x)||p(z)) = \mathbb{E}_{q_\phi(z|x)}[\log \frac{q_\phi(z|x)}{p(z)}]$. Observe that we retrieve the standard ELBO when $\beta = 1$.

This objective is useful for

- emphasizing high-fidelity reconstructions *or* disentangling the latent space
- obtaining an increased interpretability
- doing KL warm-up.

The effects of the β-objective are mostly visible asymptotically, why we may not experience the full power of it here (e.g., NVAE applies KL warm-up during the *first 25k iterations*).

However, plenty of the state-of-the-art VAEs (in terms of density estimation) apply KL warm-up (e.g. NVAE, VampPrior, LadderVAE; see LadderVAE for a nice motivation of KL warm-up), so it is good to know. Also, aside from being very straightforward to implement, the β-objective leads us to the following important note.

Important note: as pointed out by Hoffman et al., the β-objective shown above cannot be considered an evidence lower-bound. Hence, test scores based on the β-objective **should not be used for benchmarking against proper marginal log-likelihood estimates (such as the ELBO)**. If you are interested, try to reverse the derivations of the ELBO in [1] while including β.

Task 4: β-ELBO

Compute the β-objective in the following function.

Hint: this should require minimal changes to the `get_elbo` function above. Also, observe that

$\mathcal{L}_\beta \simeq \frac{1}{L}\sum_l \log p_\theta(x|z_l) - \beta \log \frac{q_\phi(z_l|x)}{p(z_l)}$.

```
@jit
def get_beta_elbo(params, x, rng, beta):
    # the negative elbo should be a scalar value, i.e. averaged over the mini-batch.
    (mu, sigma), z, bernoulli_theta = forward(params, x, rng)
    # write code to compute the ELBO




    return -beta_elbo
```

Now we define `epoch_step_beta` that calls `get_beta_elbo` instead of `get_elbo`, and so that it can take a $\beta \geq 0$ as input.

```
def epoch_step_beta(epoch, training_generator, opt_state, beta):
    # an rng, random number generator, is required for all stochastic operations
    rng = random.PRNGKey(epoch)
    nlls = []
    for i, (x, y) in enumerate(tqdm(training_generator)):
        x = jax.device_put(x) / 255
        value, grads = jax.value_and_grad(get_beta_elbo)(get_params(opt_state),
                                                         x,
                                                         rng,
                                                         beta)
        opt_state = opt_update(i,
                               grads,
                               opt_state)
        nlls.append(value)
    return opt_state, np.mean(nlls)
```

Again, importantly, we have used $\beta = 1$ as default in `evaluate_beta`, which makes the evaluation scores comparable to those of the standard VAE that we trained above.

```
def evaluate_beta(opt_state, x, test_rng,beta=1):
    params = get_params(opt_state)
    x = jax.device_put(x) / 255
    nll = get_beta_elbo(params,
                        x,
                        test_rng,beta)
    return nll
```

Before retraining the VAE, or β-VAE, we need to reinitialize the encoder/decoder nets and the optimizer above. This is done by simply running the cell below.

```
beta = 100  # just an example for a big beta. Try with other values!
latent_dimensions = 2

# encoder defs
encoder_init, encode = stax.serial(
    Dense(300), Relu,
    Dense(300), Relu,
    FanOut(2),
    stax.parallel(Dense(latent_dimensions), stax.serial(Dense(latent_dimensions), Softplus)),
)

# decoder defs
decoder_init, decode = stax.serial(
    Dense(300), Relu,
    Dense(300), Relu,
    Dense(28 * 28), Sigmoid
)


# initialize networks and get params
enc_init_key, dec_init_key = random.split(random.PRNGKey(100))
_, encoder_params = encoder_init(enc_init_key, input_shape=(-1, 28 * 28))
_, decoder_params = decoder_init(dec_init_key,
                                 input_shape=
                                 (-1, latent_dimensions))
params = encoder_params, decoder_params

# initialize optimizer
opt_init, opt_update, get_params = optimizers.adam(step_size=0.001)
opt_state = opt_init(params)
```

Rerun the experiments section below. Any difference? If you train for a few couple of epochs, then we shouldn't expect huge variations. But perhaps setting β to something very big will affect the distribution of z's in the latent space? Does setting $\beta = 0$ affect the fidelity of the reconstructed image? Why would we, at least asymptotically, expect these effects?

Task 4: β-Experiments

```
n_epochs = 20
train_key,test_key = random.split(random.PRNGKey(10))
for epoch in range(n_epochs):
    print("Epoch: ", epoch)
    opt_state, nll_train = epoch_step_beta(epoch,
                                           training_generator,
                                           opt_state,
                                           beta)
    nll_test = evaluate_beta(opt_state,
                             test_images,
                             test_key)
    print("Train NLL: ", nll_train)
    print("Test NLL: ", nll_test)
```

What has changed in the latent space? Try playing with a few values of β and explain what affects the change in the latent space?

```
# visualize the latent space
```

What about the reconstruction quality of your model? Has it improved or degraded?

```
# reconstruct an image
```

Sample from the latent space and decode. Do you see any difference?

```
# sample a few images
```

Importance Weigthed Auto-Encoder (IWAE)

Similarily as for β-VAE, the distinguishing factor between the VAE and IWAE is the alternative objective function, the importance weighted ELBO (IWELBO):

$\mathcal{L}_K = \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{1}{K}\sum_k \frac{p_\theta(x|z_k)p(z_k)}{q_\phi(z_k|x)}\right].$

When computing $\mathcal{L}_K$ in contrast to the ELBO, we sample K latent representations instead of one. Although this might seem like a simple extension, the IWELBO has huge implications on the field of VAEs, *as well as* the VI literature in general (see e.g. Domke and Sheldon). Specifically (n.b., *not an exhaustive list*),

- it is a proper lower-bound on the marginal log-likelihood.
- it is an effective way of improving the NLL test scores (used by state-of-the-art VAEs when reporting NLL scores, including those mentioned above).

The former bullet implies that we *can* use IWELBO to benchmark our VAE against other VAEs, in contrast to the β-objective. The latter bullet implies that we probably *should* use it when benchmarking. Because, accoring to Theorem 1 in the IWAE paper, the marginal log-likelihood estimates are montonically increasing with K, meaning that

$\mathcal{L}_K \geq \mathcal{L}_{K-1} \geq \mathcal{L}$.

Task 5: IWELBO

Modify the following functions such that K samples are drawn instead of 1. Then compute the IWELBO.

```
def sample_K(rng, mu, std, K):
  # use the reparameterization trick and sample K latent variables rather than 1
  return z


@partial(jax.jit, static_argnames=['K'])
def forward_K(params, x, rng, K):
    encoder_params, decoder_params = params
    phi = encode(encoder_params, inputs=x)
    z = sample_K(rng,
            mu=phi[0],
            std=phi[1],
            K=K)
    bernoulli_theta = decode(decoder_params, z)
    return phi, z, bernoulli_theta


def get_iw_elbo(params, x, rng, K):
    # the negative elbo should be a scalar value, i.e. averaged over the mini-batch.
    (mu, sigma), z, bernoulli_theta = forward_K(params, x, rng, K)
    # hint: use the logsumexp() operation over K (available in the jax library)
    # and subtract with logK to do the average inside the log.
    return -iw_elbo


def epoch_step_K(epoch, training_generator, opt_state,beta):
    # an rng, random number generator, is required for all stochastic operations
    rng = random.PRNGKey(epoch)
    nlls = []
    for i, (x, y) in enumerate(tqdm(training_generator)):
        x = jax.device_put(x) / 255
        value, grads = jax.value_and_grad(get_iw_elbo)(get_params(opt_state),
                                                       x,
                                                       rng,
                                                       K)
        opt_state = opt_update(i,
                               grads,
                               opt_state)
        nlls.append(value)
    return opt_state, np.mean(nlls)


def evaluate_K(opt_state, x, test_rng, K):
    params = get_params(opt_state)
    x = jax.device_put(x) / 255.0
    params, x = jax.lax.stop_gradient(params), jax.lax.stop_gradient(x)
    nll = get_iw_elbo(params, x, test_rng, K)
    return nll
```

Again, we need to re-initialize the network and optimizer states.

```
K = 4  # try with different values ranging from 1 to 10
latent_dimensions = 2

# encoder defs
encoder_init, encode = stax.serial(
    Dense(300), Relu,
    Dense(300), Relu,
    FanOut(2),
    stax.parallel(Dense(latent_dimensions), stax.serial(Dense(latent_dimensions), Softplus)),
)
```

```
# decoder defs
decoder_init, decode = stax.serial(
    Dense(300), Relu,
    Dense(300), Relu,
    Dense(28 * 28), Sigmoid
)


# initialize networks and get params
enc_init_key, dec_init_key = random.split(random.PRNGKey(100))
_, encoder_params = encoder_init(enc_init_key, input_shape=(-1, 28 * 28))
_, decoder_params = decoder_init(dec_init_key,
                                 input_shape=
                                 (-1, latent_dimensions))
params = encoder_params, decoder_params

# initialize optimizer
opt_init, opt_update, get_params = optimizers.adam(step_size=0.0001)  # this time we use a smaller learning rate due to numerical stabili
opt_state = opt_init(params)


n_epochs = 20
test_key = random.PRNGKey(10)
for epoch in range(n_epochs):
    print("Epoch: ", epoch)
    opt_state, nll_train = epoch_step_K(epoch,
                                        training_generator,
                                        opt_state,
                                        K)
    nll_test = evaluate_K(opt_state, test_images, test_key, K)
    print("Train NLL: ", nll_train)
    print("Test NLL: ", nll_test)
```

Plot your NLL test scores as a function of K for $K \in \{1, \ldots, 10\}$, and see if there is any improvement in NLL as K increases.

```
def plot_test_nll_vs_k():
  pass
```

Finally, it is valuable to know, how long did it take you to finish this practical?